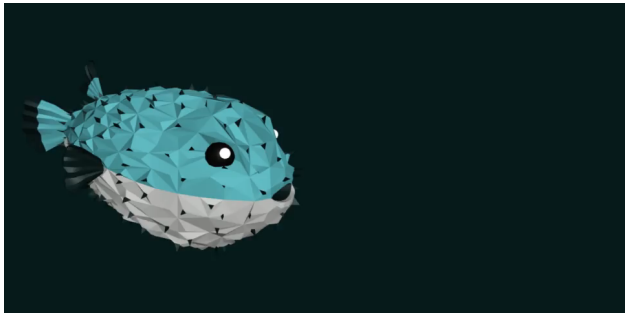
Puffing Up Reinforcement Learning

Stars 5.2k Discord 551 online Follow @jsuarez5341

Home Ocean Docs Blog



PufferLib is a library for sane and simple reinforcement learning at millions of steps per second. Our key features are:

- **Ocean:** 20+ environments from simple arcade games to massively multiagent sims
- **PuffeRL:** Train at millions of steps/second with our algorithm in a single ~1000 line script
- **Protein:** Our algorithm for automatic hyperparameter and reward tuning
- **Vectorization:** Ultra fast synchronous and asynchronous parallel simulation
- **Emulation:** Use Gymnasium and PettingZoo environments with PufferLib

These docs will get you started. Join the Discord to get help and report bugs. If you're new to RL, building and contributing a new env is the best way to learn, and we review PRs live.

Installation

Cheat Sheet

Examples

About PufferLib

Writing an Environment

FAQ

Third Party Environments

Installation

(UV) Pip

```
(uv) pip install pufferlib
(uv) pip install -e . # From source
```

- **Install nvcc** for faster training. We ship a custom advantage function kernel, and you'll only get the CPU version without it.
- **Getting PyTorch CUDA version incompatibilities? Try installing with --no-build-isolation**.
- **Install 3rd party environments** with [atari,procgen] etc. Ocean is included by default.
- **Avoid Conda** because it builds slow env code.

Docker

PufferTank is a prebuilt GPU Docker image for PufferLib. We use it for all our dev. VSCode users can install the Dev Container plugin + Docker Desktop and just open the repo. You can also set up with CLI as below. Neovim (btw) is preinstalled.

```
git clone https://github.com/pufferai/puffertank
cd puffertank
./docker.sh test
```

Cheat Sheet

```
puffer [train|eval|sweep] env_name [OPTIONS]                      # PufferLib CLI, available from package
python -m pufferlib.pufferl [train|eval|sweep] env_name [OPTIONS] # Equivalent command from source
```

```
puffer train puffer_breakout --help                          # Get help on a specific environment
puffer train puffer_breakout --train.device [cuda|cpu|mps]   # Set device. You can also set cuda:0 etc
puffer train puffer_breakout --train.learning-rate 0.001     # Set other train params
puffer train puffer_breakout --env.vision 3                  # Set env params
puffer train puffer_breakout --vec.backend Serial            # Set vec params
puffer train puffer_breakout --neptune --tag tag_name        # Track with Neptune
puffer train puffer_breakout --wandb --tag tag_name          # Track with Weights and Biases

puffer eval puffer_breakout                                  # Render the env with a random agent
puffer eval puffer_breakout --load-model-path path/model.pt  # Load a trained model
puffer eval puffer_breakout --load-model-path latest         # Load the latest model (ls -lt experimer
puffer eval puffer_breakout --neptune --load-id id           # Load a model from Neptune
puffer eval puffer_breakout --wandb --load-id id             # Load a model from WandB

puffer sweep puffer_breakout --neptune --tag tag_name        # Run a hyperparameter sweep tracked with
puffer sweep puffer_breakout --wandb --tag tag_name          # Run a hyperparameter sweep tracked with

torchrun --standalone --nnodes=1 --nproc-per-node=6 -m pufferlib.pufferl train puffer_nmmo3  # Distributed


# PuffeRL
pufferl.PuffeRL(config, vecenv, policy)                      # PuffeRL trainer
PuffeRL.evaluate()                                           # Collect batch_size environment interact
PuffeRL.train()                                              # On one batch
PuffeRL.mean_and_log()                                       # Aggregate logs and send to Wandb/Neptur
PuffeRL.close()                                              # Final logs and close envs
PuffeRL.save_checkpoint()
PuffeRL.print_dashboard()                                    # Monitor stats in your term

PuffeRL.NoLogger
PuffeRL.NeptuneLogger
PuffeRL.WandbLogger
logger.log(logs, step)
```

```
logger.close(model_path)
logger.download()

pufferl.train(env_name, args=None, vecenv=None, policy=None, logger=None)
pufferl.eval(env_name, args=None, vecenv=None, policy=None)
pufferl.sweep(env_name, args=None)

pufferl.load_config(env_name)                                    # Load args from cli + config file
pufferl.load_env(env_name, args)
pufferl.load_policy(args, vecenv)


# VecEnv
pufferlib.vector.make(env_creator_or_creators, env_args=None, env_kwargs=None, backend=PufferEnv, num_envs=
vecenv.num_envs                                                  # Number of observations returned by step
vecenv.reset()                                                   # Synchronous reset
vecenv.async_reset(seed=None)                                    # Asynchronous reset
vecenv.step(actions)                                             # Synchronous step
vecenv.send(actions)                                             # Asynchronous step - send actions to env
vecenv.recv()                                                    # Asynchronous step - receive observatior
vecenv.notify()                                                  # Call env.notify for all envs
vecenv.close()                                                   # Close all envs


# Emulation
pufferlib.emulation.GymnasiumPufferEnv(env=None, env_creator=None, env_args=[], env_kwargs={}, buf=None, se
env.render_mode                                                  # Gymnasium render mode
env.observation_space                                            # Gymnasium/PettingZoo observation space
env.action_space                                                 # Gymnasium/PettingZoo action space
env.single_observation_space                                     # Observation space for a single agent
env.single_action_space                                          # Action space for a single agent
env.seed(seed)                                                   # Legacy Gym seed method
env.reset(seed=None)                                             # Gym/Gymnasium/PettingZoo reset
```

```
env.step(action)                                                    # Gym/Gymnasium/PettingZoo step
env.render()                                                        # Gym/Gymnasium/PettingZoo render
env.close()                                                         # Gym/Gymnasium/PettingZoo close

pufferlib.emulation.PettingZooPufferEnv(env=None, env_creator=None, env_args=[], env_kwargs={}, buf=None, s
env.agents                                                          # List of agent ids
env.possible_agents                                                 # List of possible agent ids
env.done                                                            # True if all agents are done
```

Examples

Gym: Wrapping a legacy Gym environment for use with PufferLib

Gymnasium: Wrapping a Gymnasium environment for use with PufferLib

PettingZoo: Wrapping a multiagent PettingZoo environment for use with PufferLib

PufferEnv: Minimal example of our native vector env format that supports single-agent and multi-agent environments

Structured Spaces: Using PufferLib's emulation to handle non-flat observation and action spaces

Vectorization: Fast and broadly compatible serial, multiprocessed synchronous, and multiprocessed asynchronous environment simulation

Custom PufferEnv (Squared): A sample environment with implementations in C and Python. See the tutorial below for a walkthrough

Custom PufferEnv (Target): A slightly more complex, multi-agent sample environment in C. Reference this after reading the Squared walkthrough.

PufferEnv Template: Minimal working environment. Copy this as a starting point for your new environment.

PuffeRL (importable): Minimal example importing + using PuffeRL. Allows for custom models and environments without editing our training algorithm.

PuffeRL: Our main training code for PufferLib. Inspired by CleanRL - use this as a starting point for your own algo research

Atari Wrapper: How we wrap Atari for use with PufferLib

NetHack Wrapper: How we wrap NetHack for use with PufferLib

Procgen Wrapper: How we wrap Procgen for use with PufferLib

Our Default Model: Flattens observations and handles discrete, multi-discrete, and continuous action spaces. Meant as a broadly compatible starting point.

Custom Models: A collection of architectures used for our Ocean environments

About PufferLib

PufferLib is the reinforcement learning library I wish existed during my PhD. It started as a compatibility layer to make working with complex environments a breeze. Now, it's a high-performance toolkit for research and industry with optimized parallel simulation, training at millions of steps per second, and core algorithm improvements from our own research. For a fun overview of the project, watch my RL Iceberg video.

Emulation

Complex environments may have heirarchical observations and actions, variable numbers of agents, and other quirks that make them difficult to work with and incompatible with standard reinforcement learning libraries. This was the initial motivation for

PufferLib. The idea is to make every environment *emulate* the structure of simpler environments that most original RL tooling was written for. We flatten Gym/Gymnasium/PettingZoo spaces in our wrappers and unflatten them just in time for the model forward pass. This means you don't have to keep track of structured data during vectorization, in your experience buffers, or anywhere else, allowing for simpler and faster code. We also apply extra compatibility checks and pad multiagent environments to a fixed agent count. Under the hood, our wrappers maintain the Gym/Gymnasium/PettingZoo API using a stricter subset of available spaces, so you can still use them with other libraries outside of PufferLib.

Vectorization

In RL, vectorization refers to the process of simulating multiple copies of an environment in parallel. Our Multiprocessing backend is fast -- much faster than Gymnasium's in most cases. Atari runs 50-60% faster synchronous and 5x faster async by our latest benchmark, and some environments like NetHack can be 10x faster even synchronous, with no API changes. Our vectorization works on almost any Gym/Gymnasium/PettingZoo environment after applying our 1-line emulation wrapper. PufferLib outperforms other vectorization implementations using the following optimizations:

- **A Python implementation of EnvPool.** Simulates more envs than are needed per batch and returns batches of observations as soon as they are ready. Requires using the async send/recv instead of the sync step API.
- **Multiple environments per worker.** Important for fast environments.
- **Shared memory.** Unlike Gymnasium's implementation, we use a single buffer that is shared across environments.
- **Shared flags.** Workers busy-wait on an unlocked flag instead of signaling via pipes or queues. This virtually eliminates interprocess communication overhead. Pipes are used once per episode to communicate aggregated infos.
- **Zero-copy batching.** Because we use a single buffer for shared memory, we can return observations from contiguous subsets of workers without ever copying observations. Only does not work for full-async mode.
- **Native multiagent support.** It's not an extra wrapper or slow bolt-on feature. PufferLib treats single-agent and multi-agent environments the same. API differences are handled at the emulation level.

PuffeRL

PuffeRL is our training algorithm. It's based on CleanRL's PPO + LSTM but with a ton of improvements, including core algorithm changes based on our own research. We have no plans to ship a collection of standard implementations. Instead, our goal is to continue improving our one core algorithm through research and rigorous testing on all of Ocean. See our Blog for research updates!

Policies

We don't have a policy API. You just write normal PyTorch code. We do provide some defaults and surrounding tools:

- **PyTorch observation unflattening:** A batched inversion function for the flattening applied to structured observation spaces by our emulation layer
- **Default policies:** A small collection of broadly useful networks. These include MLPs and CNNs. You can also refer to all of the models used as defaults by our Ocean environments.
- **LSTM Integration:** Break your `forward()` function into `encode_observations()` and `decode_actions()` and our LSTM wrapper will handle recurrance for you. We have a major optimization here that uses LSTMCell during rollouts and LSTM during training for up to 3x faster inference.

Tutorial: Writing Your Own Environment at 1M+ SPS (steps/second/core)!

Ocean environments are written with the PufferEnv API. It's a vector format very similar to Gymnasium's VectorEnv, but with native multiagent support. Writing your own is a great way to learn and a great way to contribute to PufferLib. Follow this guide and then copy the template as a starting point. PR something cool to have it reviewed live on stream! Here's a pure Python environment using

this format: pysquared.py

If you're familiar with Gymnasium/PettingZoo, you'll notice that this is almost identical. The main difference is that observations, actions, rewards, etc. are initialized from this *buf* object. All operations happen in-place to avoid creating and copying redundant arrays. Our vectorization passes in slices of shared memory during multiprocessing, so your environment is storing observations directly to a batch on the main process. Note that this means calling *step* again will overwrite your observations etc. from the previous call.

The above environment runs at several hundred thousand steps/second, but pure Python quickly becomes a bottleneck as environments become more complex. Below is the exact same code written in C that runs over 100M sps: squared.h

Using our standard 2-core double-buffered multiprocessing settings, this environment trains at 4M steps/second while the pure Python version only trains at 400k. There are a few more steps to bind to PufferLib, though. Add a main in a separate .c so you can demo the code and work in C locally without having to compile through Python: squared.c

Now you have to bind to Python. We provide a short Python C API wrapper for this. All you have to do is write a short binding file that passes initialization args from Python and logs back from C: binding.c

You can now compile the environment as a Python C extension. If you are using a fork of PufferLib, setup.py will automatically look for *binding.c*. Ensure you get the latest build with:

```
python setup.py build_ext --inplace --force
```

To use your new environment from Python, set it up as a PufferEnv: squared.py

And finally, if you want to expose your environment to our trainer, add a line to *ocean/environment.py* and add a .ini file for your environment to *pufferlib/config/ocean*: squared.ini

Your environment is now available just like any other Ocean environment:

```
puffer train puffer_squared
```

We suggest writing your whole environment in C first. Compile with scripts/build_ocean.sh. local to enable address sanitizer. This will catch most indexing and overflow bugs. If your env works in C but not when you bind to Python, you can usually get a stack track as follows:

```
DEBUG=1 python setup.py build_ext --inplace --force
CUDA_VISIBLE_DEVICES=None LD_PRELOAD=$(gcc -print-file-name=libasan.so) python3.12 -m pufferlib.pufferl tra
```

You do have to have your compiler and asan set up correctly. This is done for you in PufferTank. You can also set breakpoints from Python into C with gdb:

```
gdb --args python3.12 -m pufferlib.pufferl train  --train.device cpu --vec.backend Serial
```

Here's a checklist of common bugs if your env is not training:

- **Zero or incorrect observations/actions:** Ensure the data type and shapes used in the Python spaces and observation/action buffers match the types you've defined in C
- **Incorrect or missing resets:** Your environment should handle its own resets internally. For envs that never reset, it is often useful to respawn agents if they are stuck (e.g. no reward for 500 steps)

- **Not zeroing observations, rewards, or terminals:** If you don't zero out rewards, they will retain their value from the previous step. Ditto for terminals. Zero them at the start of c_step to be safe. A single memset will do it for multiagent envs. If you are not setting every element of every obs (i.e. one-hots), make sure to clear those too.
- **Manually inspect data scale:** You want observations and rewards to be roughly in the range of -1 to 1.
- **Incorrect binding args:** Ensure your binding sets the same args as your .c file does. Call your init function if you have one.

There's also second tutorial environment called Target. It's multi-agent with a continuous state space and multi-discrete actions. It's commented like Squared, which is also in the repo. Star on your way in to support! When you're ready to write your own environment, you can use this template as a starting point. Set it up as follows:

```
cp -r pufferlib/ocean/template pufferlib/ocean/your_env_name
grep -rin --include=*.{h,c,py} template pufferlib/ocean/template
```

Replace instances of "template" and "Template" shown with your environment name. Also copy the .ini file from pufferlib/config/ocean/template.ini and add a line for your environment to pufferlib/ocean/environment.py. You can set up outside of PufferLib, but this lets you use our extension compilation.

FAQ

Why is it called PufferLib? Would you have rathered yet another minimal tech logo? Here, have a pufferfish 🐡

I'm new to RL. How do I contribute? Start by building a simple new environment in C and getting it to train. I review environment PRs from new contributors live on stream.

How do I export gifs? We have this build in to the demo for most envs. There's not an easy way to get frames from Ocean envs, but you can use F12 for screenshots and control+F12 to record a gif.

Multiprocessing erroring/hanging on Mac: Your OS doesn't like to run subprocesses without __main__

Third Party Environments

You can use any well-behaved environment with PufferLib via a 1-line wrapper. These are just the environments we have gotten around to binding manually. A lot of environments are not well behaved and subtly deviate from the Gymnasium/PettingZoo API, or they use obscure features that most libraries are not designed to handle. Our bindings just help clean this up a bit. Plus, we add any tricky system package dependencies to PufferTank. Feel free to PR new bindings or fixes for existing ones!

Stars 37k

OpenAI Gym is the standard API for single-agent reinforcement learning environments. It also contains some built-in environments. We include Box2D in our registry.

Stars 7.8k

Pokemon Red is one of the original Pokemon games for gameboy. This project uses the game as an environment for reinforcement learning. We are actively supporting development on this one!

Stars 3.4k

PettingZoo is the standard API for multi-agent reinforcement learning environments. It also contains some built-in environments. We include Butterfly in our registry.

Stars 2.4k

Arcade Learning Environment provides a Gym interface for classic Atari games. This is the most popular benchmark for reinforcement learning algorithms.

Stars 2.4k

Minigrid is a 2D grid-world environment engine and a collection of builtin environments. The target is flexible and computationally efficient RL research.

Stars 1.8k

MAgent is a platform for large-scale agent simulation.

Stars 1.6k

Neural MMO is a massively multiagent environment for reinforcement learning. It combines large agent populations with high per-agent complexity and is the most actively maintained (by me) project on this list.

Stars 1.1k

Procgen is a suite of arcade games for reinforcement learning with procedurally generated levels. It is one of the most computationally efficient environments on this list.

Stars 979

Nethack Learning Environment is a port of the classic game NetHack to the Gym API. It combines extreme complexity with high simulation efficiency.

Stars 511

MiniHack Learning Environment is a stripped down version of NetHack with support for level editing and custom procedural generation.

Stars 528

Crafter is a top-down 2D Minecraft clone for RL research. It provides pixel observations and relatively long time horizons.

Stars 587 GPUDrive GPUDrive is a GPU-accelerated, multi-agent driving simulator that runs at 1 million FPS.

Stars 257 Griddly is an extremely optimized platform for building reinforcement learning environments. It also includes a large suite of built-in environments.

Stars 287 Gym MicroRTS is a real time strategy engine for reinforcement learning research. The Java configuration is a bit finicky -- we're still debugging this.